

Gusto Meets — Build Guide & Strategic Blueprint

Status: Pre-build · June 2026 **Role context:** This document is authored from the combined perspective of Chief Business Director + Technical Lead for Adi (Adithya Narayanan C.), to be executed via Claude Code CLI on a personal laptop.

Part 1: Strategic verdict

The concept in one line

An asset-light, Airbnb-style marketplace PWA that lets people rent verified residential terraces by the hour for private socializing, powered by the Gusto ecosystem (wallet, KYC, booking engine).

What the model gets right

- Zero capex on supply:** You don't own terraces. You aggregate them. This is fundamentally cheaper than the lounge model (no lease, no fit-out, no AC bill).
- Cultural wedge is sharp:** The non-veg restriction at home, the "where do 6 friends meet privately for 3 hours" problem, the college-crowd-with-no-budget problem — these are all real, daily, and unsolved in Chennai.
- Turf Town validated the pattern:** Chennai already pays ₹800–₹1,500/hr for hourly turf rental. Terrace rental at ₹200–₹400/hr is the same behavior at a lower price point with a broader use case.
- Gusto synergy intact:** The terrace becomes a Gusto-powered venue — wallet, QR check-in, order-ahead food pickup nearby, all flowing through one app.

The one risk that matters

Physical safety liability. A terrace is an elevated, open-air space. Indian tort law and consumer protection law will hold the platform partially liable if a guest is injured, regardless of digital waivers. Section 79 Safe Harbor protects against *user-generated content* liability, not *physical harm* in a space you facilitated access to.

Mitigation (non-negotiable for launch):

- Phase 1 is "Curated Marketplace" — you personally inspect every terrace before it goes live. No self-serve host onboarding until you have 50+ successful bookings and a legal opinion.
- Minimum 4-foot parapet wall, verified by your team's physical inspection (not just a video upload).
- Mandatory host liability acknowledgment AND guest assumption-of-risk waiver, both drafted by a local lawyer (not a template).
- Explore group personal accident insurance (GPA) riders — ₹500–₹1,500/year per terrace, covers up to ₹5 lakh per incident. Talk to ICICI Lombard or Bajaj Allianz commercial lines.

Phase strategy

Phase	Timeline	Scope	Key metric
1. Curated MVP	Weeks 1–8	5–8 hand-picked terraces in Velachery/Perungudi/OMR. Invite-only guests	50 bookings, 80%+ satisfaction

Phase	Timeline	Scope	Key metric
(college WhatsApp groups, IT corridor circles).			
2. Open guest-side	Weeks 9–16	Open guest registration. Keep host onboarding curated. Add Gusto food ordering integration.	200 bookings/month, 15+ terraces
3. Open host-side	Weeks 17–24	Self-serve host onboarding with video verification + admin approval. Dynamic pricing.	50+ terraces, unit economics positive
4. Franchise playbook	Month 7+	Replicate in Bengaluru/Hyderabad. License the platform to city operators.	Second-city launch

Part 2: Technical architecture

Stack decisions (final)

Layer	Choice	Why
Frontend	Next.js 14 (App Router) + Tailwind + Zustand	PWA-ready, your existing skillset from Gusto menu app, SSR for SEO on listing pages
Backend/DB	Supabase (Postgres + Auth + Realtime + Edge Functions)	Eliminates need for a separate FastAPI server for 80% of flows. Phone OTP auth built in. RLS for multi-tenant isolation. Free tier = 500MB + 50k users.
Payments	Razorpay	UPI intent, pre-auth for deposits, wallet top-ups. Well-documented, India-native.
KYC	Setu (Aadhaar verification) or Zoop.one	No-OTP demographic validation. Run via Supabase Edge Function (Deno), not client-side.
Comms	WhatsApp Business API (via Interakt or Wati)	Booking confirmations, T-15 alerts, host notifications.
Hosting	Vercel (frontend) + Supabase Cloud (DB)	Both free tier. Zero infra cost for MVP.
Maps	Google Maps Platform (Places + Geocoding)	Terrace location pins, distance-from-user search.

Why NOT FastAPI as a separate backend

Your BRD spec mentions Node.js + FastAPI microservices. For Phase 1, this is overengineering. Supabase Edge Functions (Deno/TypeScript) handle the two server-side tasks that can't run client-side:

1. **Aadhaar KYC call** — a Supabase Edge Function makes the POST to Setu, runs Verhoeff validation, stores the token, and returns verified/not-verified. ~50 lines of code.
2. **Overstay penalty calculation** — a database trigger or cron function that checks `end_time` vs `NOW()`, calculates penalty, and deducts from wallet.

Everything else (CRUD on terraces, bookings, users, permissions) runs through Supabase's auto-generated REST API with Row Level Security policies. You don't need a backend server.

If you outgrow this at 500+ bookings/month, *then* extract the booking engine into a FastAPI service. Not before.

Refined database schema

The schema from your BRD is solid. Here are the refinements:

```
-- =====
-- ENUMS
-- =====
CREATE TYPE user_role AS ENUM ('GUEST', 'HOST', 'ADMIN');
CREATE TYPE access_type AS ENUM ('PRIVATE_STAIRCASE', 'SHARED_WALKTHROUGH');
CREATE TYPE booking_purpose AS ENUM (
    'CHILLOUT', 'DINE_OUT', 'MOVIE_NIGHT',
    'PARTY', 'BOARD_GAMES', 'STUDY_GROUP'
);
CREATE TYPE booking_status AS ENUM (
    'PENDING_PAYMENT', 'CONFIRMED', 'ACTIVE',
    'EXTENDED',          -- NEW: tracks mid-session extensions
    'COMPLETED', 'OVERSTAYED', 'DISPUTED', 'CANCELLED'
);
CREATE TYPE verification_status AS ENUM (
    'UNVERIFIED', 'PENDING_INSPECTION', 'VERIFIED', 'SUSPENDED'
);

-- =====
-- USERS
-- =====
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    phone_number VARCHAR(15) UNIQUE NOT NULL,
    full_name VARCHAR(100),
    role user_role DEFAULT 'GUEST',

    -- KYC (zero-knowledge: no Aadhaar stored)
    kyc_verified BOOLEAN DEFAULT FALSE,
    kyc_reference_token VARCHAR(255),
    kyc_verified_name VARCHAR(100),

    -- Wallet
    wallet_balance DECIMAL(10, 2) DEFAULT 0.00
        CHECK (wallet_balance >= 0),

    -- Trust score (built over bookings)
    completed_bookings INT DEFAULT 0,
    avg_host_rating DECIMAL(3, 2),

    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
```

```
-- TERRACES
-- =====

CREATE TABLE terraces (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  host_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

  title VARCHAR(150) NOT NULL,
  description TEXT,
  address_line TEXT NOT NULL,
  city VARCHAR(50) DEFAULT 'Chennai',
  area VARCHAR(100), -- e.g., 'Velachery', 'OMR', 'Perungudi'
  geo_lat DECIMAL(10, 7),
  geo_lng DECIMAL(10, 7),

  -- Access & capacity
  access_category access_type NOT NULL,
  max_capacity INT NOT NULL CHECK (max_capacity BETWEEN 2 AND 30),

  -- Pricing
  base_hourly_rate DECIMAL(10, 2) NOT NULL CHECK (base_hourly_rate > 0),
  min_booking_hours INT DEFAULT 1 CHECK (min_booking_hours >= 1),

  -- Safety (verified by platform team in Phase 1)
  parapet_height_ft DECIMAL(4, 2) CHECK (parapet_height_ft >= 4.0),
  safety_video_url TEXT,
  is_affidavit_signed BOOLEAN DEFAULT FALSE,
  verification verification_status DEFAULT 'UNVERIFIED',

  -- Media (array of image URLs)
  photos TEXT[],

  is_active BOOLEAN DEFAULT FALSE,

  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- PERMISSION MATRIX
-- =====

CREATE TABLE terrace_permissions (
  terrace_id UUID PRIMARY KEY REFERENCES terraces(id) ON DELETE CASCADE,

  allow_alcohol BOOLEAN DEFAULT FALSE,
  allow_smoking BOOLEAN DEFAULT FALSE,
  allow_loud_music BOOLEAN DEFAULT FALSE,
  allow_outside_food BOOLEAN DEFAULT TRUE,
  allow_couples BOOLEAN DEFAULT TRUE,
  allow_overnight BOOLEAN DEFAULT FALSE, -- future: late-night sessions

  allowed_purposes booking_purpose[] NOT NULL,

  -- Dynamic pricing multipliers
  party_multiplier DECIMAL(3, 2) DEFAULT 1.30,
```

```

    alcohol_deposit DECIMAL(10, 2) DEFAULT 500.00,

    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- BOOKINGS
-- =====
CREATE TABLE bookings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    guest_id UUID NOT NULL REFERENCES users(id),
    terrace_id UUID NOT NULL REFERENCES terraces(id),

    -- Time
    start_time TIMESTAMPTZ NOT NULL,
    end_time TIMESTAMPTZ NOT NULL,
    actual_checkout_time TIMESTAMPTZ,
    extensions_count INT DEFAULT 0,

    -- Context
    purpose booking_purpose NOT NULL,
    guest_count INT NOT NULL CHECK (guest_count > 0),
    status booking_status DEFAULT 'PENDING_PAYMENT',

    -- Financials (snapshot at booking time)
    hourly_rate_applied DECIMAL(10, 2) NOT NULL,
    total_hours DECIMAL(4, 2) NOT NULL,
    total_time_cost DECIMAL(10, 2) NOT NULL,
    security_deposit_held DECIMAL(10, 2) DEFAULT 0.00,
    platform_fee DECIMAL(10, 2) DEFAULT 0.00, -- your cut

    -- Penalties
    overstay_penalty DECIMAL(10, 2) DEFAULT 0.00,
    damage_penalty DECIMAL(10, 2) DEFAULT 0.00,

    -- Payment
    razorpay_order_id VARCHAR(100),
    razorpay_payment_id VARCHAR(100),
    digital_waiver_signed BOOLEAN DEFAULT FALSE,

    -- Host check-in confirmation
    host_checked_in BOOLEAN DEFAULT FALSE,
    host_checked_in_at TIMESTAMPTZ,

    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),

    CONSTRAINT valid_time_range CHECK (end_time > start_time)
);

-- =====
-- REVIEWS (both directions)
-- =====
CREATE TABLE reviews (

```

```

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id UUID NOT NULL REFERENCES bookings(id),
    reviewer_id UUID NOT NULL REFERENCES users(id),
    reviewee_id UUID NOT NULL REFERENCES users(id),
    rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    comment TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- WALLET TRANSACTIONS (ledger)
-- =====

CREATE TABLE wallet_transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id),
    amount DECIMAL(10, 2) NOT NULL, -- positive = credit, negative = debit
    balance_after DECIMAL(10, 2) NOT NULL,
    description VARCHAR(255),
    booking_id UUID REFERENCES bookings(id),
    razorpay_payment_id VARCHAR(100),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- DAMAGE REPORTS
-- =====

CREATE TABLE damage_reports (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id UUID NOT NULL REFERENCES bookings(id),
    host_id UUID NOT NULL REFERENCES users(id),
    description TEXT NOT NULL,
    photos TEXT[] NOT NULL,
    claimed_amount DECIMAL(10, 2),
    resolved_amount DECIMAL(10, 2),
    admin_notes TEXT,
    status VARCHAR(20) DEFAULT 'PENDING', -- PENDING, APPROVED, REJECTED
    created_at TIMESTAMPTZ DEFAULT NOW(),
    resolved_at TIMESTAMPTZ
);

-- =====
-- INDEXES
-- =====

CREATE INDEX idx Terraces_active ON terraces(is_active, city, area);
CREATE INDEX idx_bookings_overlap ON bookings(terrace_id, start_time, end_time)
    WHERE status IN ('CONFIRMED', 'ACTIVE', 'EXTENDED', 'OVERSTAYED');
CREATE INDEX idx_bookings_guest ON bookings(guest_id, status);
CREATE INDEX idx_wallet_user ON wallet_transactions(user_id, created_at DESC);

```

Key additions over the BRD schema

- **wallet_transactions table** — full ledger for every credit/debit. Essential for dispute resolution and financial auditing.
- **reviews table** — bidirectional (guest rates host, host rates guest). Builds trust layer.
- **damage_reports table** — structured dispute flow with photo evidence and admin resolution.
- **verification_status enum** — terraces go through UNVERIFIED → PENDING_INSPECTION → VERIFIED → (optionally SUSPENDED). Controls the curated onboarding flow.
- **platform_fee on bookings** — your revenue. Track it per booking for unit economics visibility.
- **extensions_count** — tracks how many times a session was extended (behavioral data for pricing optimization).
- **area column on terraces** — enables "terraces near me in Velachery" filtering without needing PostGIS for Phase 1.

Part 3: Project structure for Claude Code CLI

Directory layout

```

gusto-meets/
├── src/
│   ├── app/                                # Next.js App Router
│   │   ├── layout.tsx                      # Root layout + PWA meta
│   │   ├── page.tsx                       # Landing/search page
│   │   ├── explore/
│   │   │   └── page.tsx                   # Browse terraces with filters
│   │   ├── terrace/
│   │   │   └── [id]/
│   │   │       └── page.tsx               # Terrace detail + booking
│   │   ├── booking/
│   │   │   ├── [id]/
│   │   │   │   └── page.tsx              # Active booking (timer, extend)
│   │   │   └── history/
│   │   │       └── page.tsx              # Past bookings
│   │   ├── wallet/
│   │   │   └── page.tsx                   # Top-up, balance, transactions
│   │   ├── kyc/
│   │   │   └── page.tsx                   # Aadhaar verification flow
│   │   ├── host/
│   │   │   ├── dashboard/
│   │   │   │   └── page.tsx              # Host: incoming bookings, check-in
│   │   │   └── listing/
│   │   │       └── page.tsx              # Host: manage terrace listing
│   │   └── admin/
│   │       └── page.tsx                   # Admin: verify terraces, disputes
│   ├── components/
│   │   ├── ui/                            # Reusable: Button, Card, Modal, Timer
│   │   ├── booking/                       # BookingCard, TimeSlotPicker, ExtendModal
│   │   ├── terrace/                       # TerraceCard, PermissionBadges, PhotoGallery
│   │   ├── wallet/                        # WalletBalance, TopUpSheet, TransactionRow
│   │   └── layout/                        # Navbar, BottomNav, SearchBar
│   ├── lib/
│   └── supabase/

```

```

├── client.ts      # Browser Supabase client
├── server.ts      # Server-side Supabase client
├── types.ts       # Generated TypeScript types from schema
├── razorpay.ts    # Payment helper functions
├── utils.ts       # Verhoeff check, formatCurrency, etc.
├── constants.ts   # Pricing rules, platform fee %, etc.
├── store/
│   └── useBookingStore.ts # Zustand: active booking, wallet, timer
├── styles/
│   └── globals.css      # Tailwind base + custom properties
├── supabase/
│   ├── migrations/
│   │   └── 001_initial_schema.sql # The schema above
│   ├── functions/
│   │   ├── verify-aadhaar/
│   │   │   └── index.ts          # Edge function: KYC API call
│   │   └── check-overstays/
│   │       └── index.ts          # Edge function: cron for penalty calc
│   └── seed.sql                 # 5-8 test terraces for dev
├── public/
│   ├── manifest.json           # PWA manifest
│   ├── sw.js                   # Service worker
│   └── icons/                   # PWA icons
├── next.config.js
├── tailwind.config.ts
├── tsconfig.json
└── package.json

```

Build sequence (Claude Code CLI prompts)

Feed these into Claude Code CLI in order. Each prompt is a self-contained task.

Prompt 1 — Scaffold + DB

Initialize a Next.js 14 project with App Router, TypeScript, and Tailwind CSS in the current directory. Set up Supabase client libraries (@supabase/supabase-js, @supabase/ssr). Create the directory structure as specified in the project layout. Create the Supabase migration file at supabase/migrations/001_initial_schema.sql with the full schema including all tables (users, terraces, terrace_permissions, bookings, reviews, wallet_transactions, damage_reports), enums, and indexes. Create a seed.sql with 3 sample terraces in Velachery, OMR, and Perungudi with realistic Chennai addresses and coordinates. Set up the PWA manifest.json and a basic service worker. Configure environment variables in .env.local for NEXT_PUBLIC_SUPABASE_URL and NEXT_PUBLIC_SUPABASE_ANON_KEY.

Prompt 2 — Auth + KYC flow

Build the authentication flow using Supabase Auth with phone number + OTP. Create the login page at `src/app/page.tsx` with a phone input and OTP verification step. After auth, check if the user exists in the users table; if not, insert them. Create the KYC page at `src/app/kyc/page.tsx` with:

- (1) Aadhaar number input with client-side Verhoeff checksum validation,
- (2) full name input, (3) a submit button that calls a Supabase Edge Function.

Create the edge function at `supabase/functions/verify-aadhaar/index.ts` that accepts `{ aadhaar, name }`, calls the Setu sandbox API for No-OTP verification, performs fuzzy name matching, and updates the user's `kyc_verified`, `kyc_reference_token`, and `kyc_verified_name` fields. Show success/failure state on the KYC page. Use Zustand store for auth state.

Prompt 3 — Explore + terrace detail

Build the terrace discovery flow. Create `src/app/explore/page.tsx` that:

- (1) fetches active, verified terraces from Supabase filtered by `city='Chennai'`,
- (2) shows them as cards with photo, title, area, `base_hourly_rate`, capacity, and permission badges (icons for alcohol/smoking/music/food allowed),
- (3) has filter buttons for area (Velachery, OMR, Perungudi, All) and purpose (Chillout, Dine Out, Party, etc). Create the terrace detail page at `src/app/terrace/[id]/page.tsx` that shows: full photo gallery, description, host name and rating, permission matrix, pricing breakdown (base rate + multipliers), a date/time slot picker for booking, and a "Book Now" CTA. The slot picker must check existing bookings to prevent double-booking (query bookings table for overlapping time ranges). Include a 30-minute buffer between bookings. Mobile-first responsive design using Tailwind.

Prompt 4 — Booking + payment

Build the booking and payment flow. When user taps "Book Now" on the terrace detail page:

- (1) calculate total cost = `hourly_rate * hours * purpose_multiplier + security_deposit + platform_fee (10%)`,
- (2) if user has sufficient wallet balance, debit from wallet and create booking with status `CONFIRMED`,
- (3) if insufficient balance, initiate Razorpay checkout for the difference (use Razorpay test mode),
- (4) on payment success, create the booking record, debit wallet, record `wallet_transaction` entries,
- (5) redirect to the active booking page at `src/app/booking/[id]/page.tsx`.

The active booking page shows: a live countdown timer (Zustand store), terrace address with a "Navigate" button (Google Maps intent link), host contact, and the check-in status. At T-minus 15 minutes, show an "Extend Session" modal with options for 30min or 1hr extension. Extension debits from wallet or triggers Razorpay. Create the wallet page at `src/app/wallet/page.tsx` with balance display, top-up via Razorpay, and transaction history from `wallet_transactions` table.

Prompt 5 — Host dashboard

Build the host-side experience. Create `src/app/host/dashboard/page.tsx` that shows: (1) today's incoming bookings as cards with guest name, guest_count, purpose, time slot, and KYC verified badge, (2) a "Check In Guest" button that updates `host_checked_in=true` and sets booking status to ACTIVE, (3) currently active bookings with a live timer showing remaining time, (4) a "Report Damage" button on completed bookings that opens a form to upload photos and describe damage (creates a `damage_report` record and freezes the security deposit). Use Supabase Realtime to subscribe to new bookings for this host's terraces – show a toast/ping when a new booking comes in. Create `src/app/host/listing/page.tsx` for the host to view and edit their terrace details and permission matrix. For Phase 1, host onboarding is manual (admin creates the listing), so this page is view/edit only, not create.

Prompt 6 — Admin panel + overstay handling

Build the admin panel at `src/app/admin/page.tsx` with three tabs: (1) Terrace Verification – list of terraces with `verification_status` PENDING_INSPECTION, show safety video link, parapet height, affidavit status. Admin can approve (set to VERIFIED, `is_active=true`) or reject (set to SUSPENDED with a note). (2) Damage Disputes – list of pending `damage_reports` with booking context, guest/host info, photos. Admin can approve (capture penalty from deposit, route to host wallet) or reject (refund deposit to guest). (3) Active Overstays – list of bookings where `NOW() > end_time` AND `status = ACTIVE`. Create the Supabase Edge Function at `supabase/functions/check-overstays/index.ts` that runs on a cron schedule (every 5 minutes), finds overstayed bookings, updates status to OVERSTAYED, calculates penalty (2x hourly rate per extra hour), deducts from `security_deposit_held`, and creates a `wallet_transaction`.

Part 4: Revenue model for Phase 1

Per-booking unit economics (example)

Line item	Amount	Who pays
Base terrace rate (2 hrs × ₹200/hr)	₹400	Guest
Party multiplier (1.3×)	₹520 total	Guest
Platform fee (15% of time cost)	₹78	Deducted from host payout
Security deposit (refundable)	₹500	Guest (held in escrow)
Guest pays	₹1,020	
Host receives	₹442	₹520 – ₹78
Platform keeps	₹78	

Monthly projection (Phase 1: 5 terraces)

Metric	Conservative	Optimistic
Bookings/terrace/week	4	8
Total bookings/month	80	160
Avg platform fee/booking	₹75	₹90
Monthly platform revenue	₹6,000	₹14,400
Monthly operating cost (Supabase, Vercel, WhatsApp API)	~₹2,000	~₹2,000

Phase 1 won't be profitable — that's fine. The goal is 50 bookings to validate demand. Revenue scales nonlinearly once you cross 20+ terraces (no additional platform cost per terrace).

Part 5: What to do on Monday

- 1. Personal laptop setup:** `npx create-next-app@latest gusto-meets --typescript --tailwind --app` and install Supabase CLI.
- 2. Supabase project:** Create a free project at supabase.com. Run the migration SQL. Seed the test data.
- 3. Razorpay test account:** Sign up at dashboard.razorpay.com. Get test API keys.
- 4. Paste Prompt 1** into Claude Code CLI from the `gusto-meets` directory. Review the output, commit.
- 5. Paste Prompt 2.** Build auth + KYC. This is the hardest flow — get it right before moving on.
- 6. By end of week 1:** You should have auth, KYC, and the explore page working against real Supabase data.

Dev environment note

Your dad's office system with blocked uploads is not part of this workflow. All development happens on your personal laptop → pushes to a private GitHub repo → deploys to Vercel via GitHub integration. If you want your dad to review progress, share the Vercel preview URL — no code transfer needed.

Part 6: Comparison to reference models

Dimension	Airbnb	Zoomcar	Turf Town	Gusto Meets
Supply type	Homes/rooms	Cars	Purpose-built turfs	Residential terraces
Liability model	AirCover (\$1M insurance)	Motor insurance (IRDAI)	Owned asset, controlled	Gap: no insurance product exists yet
KYC	Government ID upload	DL verification	None (walk-in)	No-OTP Aadhaar + host visual verify
Pricing	Per night	Per hour/day	Per hour	Per hour
Damage handling	Host claims → Airbnb arbitrates	Pre-auth deposit → auto-deduct	On-site staff manages	Pre-auth deposit → admin arbitrates

Dimension	Airbnb	Zoomcar	Turf Town	Gusto Meets
Trust mechanism	Reviews + Superhost	Trust score + deposit	Reputation/repeat	Reviews + KYC badge + deposit
Cold start strategy	Craigslist cross-posting	Fleet seeding	Location-based marketing	Curated 5–8 terraces, WhatsApp group seeding

Key learning from each

- **From Airbnb:** The host onboarding quality determines platform trust. Bad listings kill marketplaces. Your Phase 1 curation (personal inspection) is correct.
- **From Zoomcar:** The deposit + auto-penalty model eliminates host-guest confrontation. Your "extend or evacuate" flow is directly inspired by this and it works.
- **From Turf Town:** Chennai millennials absolutely will pay for hourly space rental. Your price point (₹200–400/hr) is lower than turfs (₹800–1500/hr), which means the affordability barrier is even lower.